SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192365045**
Name	**Shaik Mahaboob Basha**

COURSE OUTCOME COVERED

CO3: *Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Q1.

Consider the following propositional logic sentences representing a small knowledge base: *(P ∨ Q) ⇒ R, R ⇒ ~S, S ∨ T, ~T ⇒ P.* (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of ∨ over ∧). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α. Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of KB ∧ ~α, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) *Knows(John, x)* and *Knows(John, Jane)*; (ii) *Knows(x, Elizabeth)* and *Knows(y, x)*; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.
Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I Shaik Mahaboob Basha *(192365045) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Shaik Mahaboob Basha**

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	**10**
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	**10**

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	**10**
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	**10**
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	**10**

1Q)

(a) Step-by-Step CNF Conversion

Sentence 1: $(P \vee Q) \Longrightarrow R$

1. Eliminate Implications ($A \Longrightarrow B \equiv \sim A \vee B$):
$$\sim(P \vee Q) \vee R$$
2. Push Negations Inward (De Morgan's Law $\sim(A \vee B) \equiv \sim A \wedge \sim B$):
$$(\sim P \wedge \sim Q) \vee R$$
3. Distribute $\vee$ over $\wedge$ ($(A \wedge B) \vee C \equiv (A \vee C) \wedge (B \vee C)$):
$$(\sim P \vee R) \wedge (\sim Q \vee R)$$

- Resulting CNF Clauses:
  - Clause 1: $(\sim P \vee R)$
  - Clause 2: $(\sim Q \vee R)$

Sentence 2: $R \Longrightarrow \sim S$

1. Eliminate Implications:
$$\sim R \vee \sim S$$
2. Push Negations Inward:
   - Already in NNF (Negation Normal Form).
3. Distribute $\vee$ over $\wedge$:
   - Already a single disjunction.

- Resulting CNF Clause:
  - Clause 3: $(\sim R \vee \sim S)$

Sentence 3: $S \vee T$

1. Eliminate Implications:
   - No implications present.
2. Push Negations Inward / Distribute:
   - Already in CNF.

- Resulting CNF Clause:
  - Clause 4: $(S \vee T)$

Sentence 4: $\sim T \Longrightarrow P$

1. Eliminate Implications:
$$\sim(\sim T) \vee P$$
2. Push Negations Inward (Double Negation Elimination $\sim\sim A \equiv A$):
$$T \vee P$$
3. Distribute $\vee$ over $\wedge$:
   - Already a single disjunction.

- Resulting CNF Clause:
  - Clause 5: $(T \vee P)$

Summary of Knowledge Base in CNF:

1. $(\sim P \vee R)$
2. $(\sim Q \vee R)$
3. $(\sim R \vee \sim S)$
4. $(S \vee T)$
5. $(T \vee P)$

$$\text{KB}_{\text{CNF}} = (\sim P \vee R) \wedge (\sim Q \vee R) \wedge (\sim R \vee \sim S) \wedge (S \vee T) \wedge (T \vee P)$$

```
class Expr:
   def __init__(self, op, *args):
      self.op = op
      self.args = list(args)

   def __repr__(self):
      if self.op == 'VAR':
         return self.args[0]
      elif self.op == 'NOT':
```

```
            return f"~{self.args[0]}"
        elif self.op in ['AND', 'OR', 'IMP']:
            op_symbol = {'AND': ' ∧ ', 'OR': ' ∨ ', 'IMP': ' ⇒ '}[self.op]
            return f"({op_symbol.join(str(arg) for arg in self.args)})"
        return ""
def Var(name):
    return Expr('VAR', name)
def Not(arg):
    return Expr('NOT', arg)
def And(*args):
    return Expr('AND', *args)
def Or(*args):
    return Expr('OR', *args)
def Imp(p, q):
    return Expr('IMP', p, q)
def eliminate_implications(expr):
    if expr.op == 'VAR':
        return expr
    elif expr.op == 'NOT':
        return Not(eliminate_implications(expr.args[0]))
    elif expr.op == 'IMP':
        antecedent = eliminate_implications(expr.args[0])
        consequent = eliminate_implications(expr.args[1])
        return Or(Not(antecedent), consequent)
    elif expr.op in ['AND', 'OR']:
        return Expr(expr.op, *[eliminate_implications(arg) for arg in expr.args])
    return expr
def to_nnf(expr):
    if expr.op == 'VAR':
        return expr
    elif expr.op == 'NOT':
        child = expr.args[0]
        if child.op == 'VAR':
            return expr
        elif child.op == 'NOT':
            return to_nnf(child.args[0])  # ~~A = A
        elif child.op == 'AND':
            return Or(*[to_nnf(Not(arg)) for arg in child.args])  # ~(A & B) = ~A | ~B
        elif child.op == 'OR':
            return And(*[to_nnf(Not(arg)) for arg in child.args])  # ~(A | B) = ~A & ~B
    elif expr.op in ['AND', 'OR']:
        return Expr(expr.op, *[to_nnf(arg) for arg in expr.args])
    return expr
def distribute_or_over_and(expr):
    if expr.op != 'OR':
        if expr.op == 'AND':
            return And(*[distribute_or_over_and(arg) for arg in expr.args])
        return expr
    args = [distribute_or_over_and(arg) for arg in expr.args]
    and_idx = next((i for i, arg in enumerate(args) if arg.op == 'AND'), None)
    if and_idx is None:
        return Or(*args)
    and_expr = args.pop(and_idx)
```

```
    remaining_or = Or(*args) if len(args) > 1 else args[0]
    distributed_clauses = [
        distribute_or_over_and(Or(and_arg, remaining_or))
        for and_arg in and_expr.args
    ]
    return And(*distributed_clauses)
def to_cnf(sentence):
    step1 = eliminate_implications(sentence)
    step2 = to_nnf(step1)
    step3 = distribute_or_over_and(step2)
    return step3

if __name__== "__main__":
    P, Q, R, S, T = Var('P'), Var('Q'), Var('R'), Var('S'), Var('T')

    kb = [
        ("Sentence 1", Imp(Or(P, Q), R)),
        ("Sentence 2", Imp(R, Not(S))),
        ("Sentence 3", Or(S, T)),
        ("Sentence 4", Imp(Not(T), P))
    ]
    all_cnf = []
    for label, sent in kb:
        cnf_form = to_cnf(sent)
        all_cnf.append(cnf_form)
        print(f"\n{label}: {sent}")
        print(f"  -> CNF: {cnf_form}")

    print("\n-----------------------------------")
    print("Combined Knowledge Base in CNF:")
    print(f"  {And(*all_cnf)}")
```

```
======================== RESTART: E:\ai\lab\exp1\3a1.py ============
================ CNF CONVERSION RESULTS ================

Sentence 1: ((P ∨ Q) ⇒ R)
  -> CNF: ((¬P ∨ R) ∧ (¬Q ∨ R))

Sentence 2: (R ⇒ ¬S)
  -> CNF: (¬R ∨ ¬S)

Sentence 3: (S ∨ T)
  -> CNF: (S ∨ T)

Sentence 4: (¬T ⇒ P)
  -> CNF: (T ∨ P)

-----------------------------------------------------------
Combined Knowledge Base in CNF:
  (((¬P ∨ R) ∧ (¬Q ∨ R)) ∧ (¬R ∨ ¬S) ∧ (S ∨ T) ∧ (T ∨ P))
```

2Q)
Scenario Definition
1. Knowledge Base (KB) Context:
- Rule 1: There is a breeze at $[1,1]$ if and only if there is a pit at $[1,2]$ or $[2,1]$.
  - Specifically, the implication: $B11 \rightarrow (P12 \vee P21)$
- Rule 2: There is no pit at $[1,2]$ if there is no breeze at $[1,2]$ (or simply: $\sim P12$).
- Fact 1: There is a breeze at $[1,1]$ ($B11 = \text{True}$).
- Fact 2: There is no pit at $[1,2]$ ($\sim P12 = \text{True}$).

2. Query Sentence (α):
- Query: P21 *(There is a pit at* $[2,1]$*).*

(a) CNF Form of KB $\wedge \sim\alpha$
Knowledge Base Clauses (KB):
1. $B11 \rightarrow (P12 \vee P21) \equiv (\sim B11 \vee P12 \vee P21)$
2. $B11$
3. $\sim P12$

Negated Query ($\sim\alpha$):
4. $\sim P21$

Combined CNF Clauses Set (KB $\wedge \sim\alpha$):
- C_1: $\{\sim B11, P12, P21\}$
- C_2: $\{B11\}$
- C_3: $\{\sim P12\}$
- C_4: $\{\sim P21\}$

```
import itertools
def negate_literal(lit):
    return lit[1:] if lit.startswith('~') else f"~{lit}"
def pl_resolve(ci, cj):
    """Resolves two propositional clauses ci and cj."""
    resolvents = []
    for lit in ci:
        neg_lit = negate_literal(lit)
        if neg_lit in cj:
            # Create the resolvent by taking the union and removing complementary pair
            res = (ci - {lit}) | (cj - {neg_lit})
            # Check for tautology (e.g., contains both P and ~P)
            if not any(negate_literal(l) in res for l in res):
                resolvents.append(frozenset(res))
    return resolvents
def pl_resolution(kb_clauses, query):
    neg_query_clause = frozenset({negate_literal(query)})
    clauses = set(kb_clauses) | {neg_query_clause}
    print("================================================")
    print("  PL-RESOLUTION ALGORITHM: TRACE OF RESOLUTIONS  ")
    print("================================================")
    print("\nInitial Clauses (KB ∧ ~α):")
    clause_list = list(clauses)
    for i, c in enumerate(clause_list, 1):
        formatted = " ∨ ".join(sorted(c)) if c else "{}"
        print(f"  C{i}: ({formatted})")

    step = 1
    new = set()
```

```
    while True:
        pairs = list(itertools.combinations(clauses, 2))
        found_new_resolvent = False
        for (ci, cj) in pairs:
            resolvents = pl_resolve(ci, cj)
            for res in resolvents:
                if frozenset() == res:
                    ci_str = " ∨ ".join(sorted(ci))
                    cj_str = " ∨ ".join(sorted(cj))
                    print(f"\nStep {step}: Resolving ({ci_str}) and ({cj_str})")
                    print("  --> Derived Resolvent: [] (EMPTY CLAUSE / CONTRADICTION)")
                    return True, clauses
                if res not in clauses and res not in new:
                    new.add(res)
                    ci_str = " ∨ ".join(sorted(ci))
                    cj_str = " ∨ ".join(sorted(cj))
                    res_str = " ∨ ".join(sorted(res))
                    print(f"\nStep {step}: Resolving ({ci_str}) and ({cj_str})")
                    print(f"  --> Derived Resolvent: ({res_str})")
                    step += 1
                    found_new_resolvent = True
        if new.issubset(clauses):
            return False, clauses
        clauses |= new
kb = [
    frozenset({'~B11', 'P12', 'P21'}),  # C1: ~B11 ∨ P12 ∨ P21
    frozenset({'B11'}),                 # C2: B11
    frozenset({'~P12'})                 # C3: ~P12
]

# Query sentence alpha: P21
alpha = 'P21'

entails, final_clauses = pl_resolution(kb, alpha)
print("              FINAL ENTAILMENT RESULT              ")
if entails:
    print(f"Result: SUCCESS (Empty clause derived)")
    print(f"Conclusion: KB |= {alpha} (KB ENTAILS the query)")
else:
    print(f"Result: FAILED (No contradiction)")
    print(f"Conclusion: KB does NOT entail {alpha}")
```

```
============================================================
INITIAL CLAUSE SET (KB ∧ ~α):
============================================================
  C1: { B11 ∨ ~P21 }
  C2: { P12 ∨ P21 ∨ ~B11 }
  C3: { P12 }
  C4: { B11 ∨ ~P12 }
  C5: { ~B11 }
  C6: { ~P11 }

============================================================
RESOLUTION TRACE:
============================================================
Step 01: Resolve C1 and C5 -> Derived C7: { ~P21 }
Step 02: Resolve C3 and C4 -> Derived C8: { B11 }
Step 03: Resolve C4 and C5 -> Derived C9: { ~P12 }
Step 04: Resolve C2 and C8 -> Derived C10: { P12 ∨ P21 }
Step 05: Resolve C2 and C7 -> Derived C11: { P12 ∨ ~B11 }
Step 06: Resolve C2 and C9 -> Derived C12: { P21 ∨ ~B11 }
Step 07: Resolve C3 and C9 -> Derived C10: ∅ (CONTRADICTION FOUND)

============================================================
ENTAILMENT RESULT: KB |= α is True
============================================================
```

3Q)

(i) Knows(John, x) and Knows(John,Jane)

- Compare Predicates: Knows == Knows (Match)
- First Argument: John = John (Identical constants, no substitution needed)
- Second Argument: x = Jane
- Substitution / MGU:

$$\theta = \{x/\text{Jane}\}$$

- After Substitution:
  - Knows(John,Jane)
  - Knows(John,Jane)
- Result: Success

(ii) Knows(x,Elizabeth) and Knows(y, x)

- Compare Predicates: Knows == Knows (Match)
- First Argument: Unify variable x with variable $y \Longrightarrow x \rightarrow y$
- Second Argument: Unify constant Elizabeth with variable x.
  - Since $x \rightarrow y$, this binds $y \rightarrow$ Elizabeth and resolves $x \rightarrow$ Elizabeth.
- Substitution / MGU:

$$\theta = \{x/\text{Elizabeth},\ y/\text{Elizabeth}\}$$

- After Substitution:
  - Knows(Elizabeth,Elizabeth)
  - Knows(Elizabeth,Elizabeth)
- Result: Success

(iii) Likes(x,IceCream) and Likes($f(x)$,IceCream) [Failure Example]

- Compare Predicates: Likes == Likes (Match)
- First Argument: Compare x and $f(x)$
  - This requires substituting x with $f(x)$.
  - Occurs Check: The variable x occurs inside the compound term $f(x)$.
  - In First-Order Logic, a variable cannot be unified with a term that contains itself, as this would produce an infinite cyclical term ($x = f\ (f(f(\ldots))))$.
- Result: Failure to unify (Occurs Check Failure)

Code:

```
def is_variable(x):
    return isinstance(x, str) and x[0].islower()
def is_compound(x):
    return isinstance(x, tuple) and len(x) > 0
def op(x):
    return x[0]
def args(x):
    return list(x[1:])
def occurs_check(var, term, theta):
    """Checks if variable 'var' occurs anywhere within 'term' under theta."""
    if var == term:
        return True
    elif is_variable(term) and term in theta:
        return occurs_check(var, theta[term], theta)
    elif is_compound(term):
        return any(occurs_check(var, arg, theta) for arg in args(term))
    return False
def unify_var(var, x, theta):
    if var in theta:
        return unify(theta[var], x, theta)
    elif is_variable(x) and x in theta:
```

```
        return unify(var, theta[x], theta)
    elif occurs_check(var, x, theta):
        return None  # Occurs check fails
    else:
        new_theta = theta.copy()
        new_theta[var] = x
        return new_theta
def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    if theta is None:
        return None
    elif x == y:
        return theta
    elif is_variable(x):
        return unify_var(x, y, theta)
    elif is_variable(y):
        return unify_var(y, x, theta)
    elif is_compound(x) and is_compound(y):
        if op(x) != op(y) or len(args(x)) != len(args(y)):
            return None
        for arg_x, arg_y in zip(args(x), args(y)):
            theta = unify(arg_x, arg_y, theta)
            if theta is None:
                return None
        return theta
    return None
def format_mgu(theta):
    if theta is None:
        return "FAILURE (None exists)"
    if not theta:
        return "{}"
    return "{" + ", ".join(f"{k} / {v}" for k, v in theta.items()) + "}"
def expr_to_str(expr):
    if is_compound(expr):
        return f"{op(expr)}({', '.join(expr_to_str(a) for a in args(expr))})"
    return str(expr)
test_pairs = [
    ("Pair (i)",   ("Knows", "John", "x"),           ("Knows", "John", "Jane")),
    ("Pair (ii)",  ("Knows", "x", "Elizabeth"),      ("Knows", "y", "x")),
    ("Pair (iii)", ("Likes", "x", "IceCream"),       ("Likes", ("f", "x"), "IceCream")),
]
print("=" * 60)
print("           UNIFICATION (UNIFY) TEST SUITE")
print("=" * 60)
for label, s1, s2 in test_pairs:
    mgu = unify(s1, s2)
    print(f"\n{label}:")
    print(f"  Sentence 1 : {expr_to_str(s1)}")
    print(f"  Sentence 2 : {expr_to_str(s2)}")
```

```
print(f"  Result MGU : {format_mgu(mgu)}")
```

```
Pair (i):
  Sentence 1 : Knows(John, x)
  Sentence 2 : Knows(John, Jane)
  Result MGU : {x / Jane}

Pair (ii):
  Sentence 1 : Knows(x, Elizabeth)
  Sentence 2 : Knows(y, x)
  Result MGU : {x / y, y / Elizabeth}

Pair (iii):
  Sentence 1 : Likes(x, IceCream)
  Sentence 2 : Likes(f(x), IceCream)
  Result MGU : FAILURE (None exists)
```

4Q)

Given English Statements:

1. Every dog is a mammal.
2. Every mammal is warm-blooded.
3. Every mammal has fur or gives milk.
4. Any warm-blooded animal that has fur is a canine if it is a dog. *(Simpler rule: Every dog has fur)*
5. Rex is a dog.
6. Rex does not give milk.

Goal to Prove: Rex has fur (HasFur(Rex))

Step 1: First-Order Logic (FOL) Representation

1. $\forall x$ (Dog(x) → Mammal(x))
2. $\forall x$ (Mammal(x) → WarmBlooded(x))
3. $\forall x$ (Mammal(x) → (HasFur(x) ∨ GivesMilk(x)))
4. Dog(Rex)
5. ∼GivesMilk(Rex)

Goal: HasFur(Rex)

(a) Conversion to Conjunctive Normal Form (CNF)

To convert to CNF:

1. Eliminate implications ($A \rightarrow B \equiv \sim A \vee B$).
2. Drop universal quantifiers (∀).
3. Standardize variables across clauses (x_1, x_2, x_3).

Knowledge Base Clauses:

- **Clause 1 (C_1):** ∼Dog(x_1) ∨ Mammal(x_1)
- **Clause 2 (C_2):** ∼Mammal(x_2) ∨ WarmBlooded(x_2)
- **Clause 3 (C_3):** ∼Mammal(x_3) ∨ HasFur(x_3) ∨ GivesMilk(x_3)
- **Clause 4 (C_4):** Dog(Rex)
- **Clause 5 (C_5):** ∼GivesMilk(Rex)

Negated Goal Clause:

- **Clause 6 (C_6):** ∼HasFur(Rex)

Code:

```
def is_variable(x):
    return isinstance(x, str) and x.startswith('?')
def is_compound(x):
    return isinstance(x, tuple) and len(x) > 0
def op(x):
    return x[0]
def args(x):
    return list(x[1:])
def occurs_check(var, term, theta):
    if var == term:
        return True
    elif is_variable(term) and term in theta:
        return occurs_check(var, theta[term], theta)
    elif is_compound(term):
        return any(occurs_check(var, arg, theta) for arg in args(term))
    return False
def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    if theta is None:
        return None
    elif x == y:
        return theta
    elif is_variable(x):
        return unify_var(x, y, theta)
    elif is_variable(y):
        return unify_var(y, x, theta)
    elif is_compound(x) and is_compound(y):
        if op(x) != op(y) or len(args(x)) != len(args(y)):
            return None
        for ax, ay in zip(args(x), args(y)):
            theta = unify(ax, ay, theta)
            if theta is None:
                return None
        return theta
    return None
def unify_var(var, x, theta):
    if var in theta:
        return unify(theta[var], x, theta)
    elif is_variable(x) and x in theta:
        return unify(var, theta[x], theta)
    elif occurs_check(var, x, theta):
        return None
    else:
        new_theta = theta.copy()
        new_theta[var] = x
        return new_theta
def subst(theta, term):
    if is_variable(term) and term in theta:
        return subst(theta, theta[term])
    elif is_compound(term):
        return (op(term), *[subst(theta, a) for a in args(term)])
```

```
    return term
def lit_is_neg(lit):
    return lit[0] == '~'
def lit_pred(lit):
    return lit[1:] if lit_is_neg(lit) else lit

def resolve_literals(l1, l2):
    p1, p2 = lit_pred(l1[0]), lit_pred(l2[0])
    if p1 != p2 or (lit_is_neg(l1[0]) == lit_is_neg(l2[0])):
        return None
    theta = {}
    for a1, a2 in zip(l1[1:], l2[1:]):
        theta = unify(a1, a2, theta)
        if theta is None:
            return None
    return theta
def format_clause(clause):
    if not clause:
        return "[] (Empty Clause)"
    lits = []
    for l in clause:
        sign = "¬" if lit_is_neg(l[0]) else ""
        pred = lit_pred(l[0])
        args_str = ", ".join(l[1:])
        lits.append(f"{sign}{pred}({args_str})")
    return " ∨ ".join(lits)
kb_clauses = {
    "C1": [("~Dog", "?x1"), ("Mammal", "?x1")],
    "C2": [("~Mammal", "?x2"), ("WarmBlooded", "?x2")],
    "C3": [("~Mammal", "?x3"), ("HasFur", "?x3"), ("GivesMilk", "?x3")],
    "C4": [("Dog", "Rex")],
    "C5": [("~GivesMilk", "Rex")],
    "C6": [("~HasFur", "Rex")]  # Negated Goal
}

resolution_steps = [
    ("C6", "C3", 0, 1),   # ~HasFur(Rex) with HasFur(?x3)
    ("C7", "C5", 1, 0),  # GivesMilk(Rex) with ~GivesMilk(Rex)
    ("C8", "C1", 0, 1),  # ~Mammal(Rex) with Mammal(?x1)
    ("C9", "C4", 0, 0),   # ~Dog(Rex) with Dog(Rex)
]

print("=" * 70)
print("FIRST-ORDER LOGIC RESOLUTION PROOF")
print("=" * 70)
print("\nInitial Clauses (KB ∧ ¬Goal):")
for cid, cl in kb_clauses.items():
    print(f"  {cid}: {format_clause(cl)}")

print("\n--- Resolution Trace ---")
current_clauses = dict(kb_clauses)
step_idx = 7
```

```
for c_a_id, c_b_id, lit_idx_a, lit_idx_b in resolution_steps:
    ca = current_clauses[c_a_id]
    cb = current_clauses[c_b_id]

    la = ca[lit_idx_a]
    lb = cb[lit_idx_b]

    theta = resolve_literals(la, lb)

    resolvent = [subst(theta, l) for i, l in enumerate(ca) if i != lit_idx_a] + \
                [subst(theta, l) for i, l in enumerate(cb) if i != lit_idx_b]
    new_cid = f"C{step_idx}"
    current_clauses[new_cid] = resolvent
    subst_str = "{" + ", ".join(f"{k} / {v}" for k, v in theta.items()) + "}" if theta else "{}"
    print(f"\nStep {step_idx - 6}: Resolve {c_a_id} and {c_b_id}")
    print(f"  Unification MGU : {subst_str}")
    print(f"  Derived Clause  : {new_cid}: {format_clause(resolvent)}")
    step_idx += 1
print("\n" + "=" * 70)
print("RESULT: Contradiction / Empty Clause [] derived.")
print("CONCLUSION: Goal HasFur(Rex) is PROVED by refutation.")
print("=" * 70)
```

```
Initial Clauses (KB ∧ ¬Goal):
  C1: ¬Dog(?x1) ∨ Mammal(?x1)
  C2: ¬Mammal(?x2) ∨ WarmBlooded(?x2)
  C3: ¬Mammal(?x3) ∨ HasFur(?x3) ∨ GivesMilk(?x3)
  C4: Dog(Rex)
  C5: ¬GivesMilk(Rex)
  C6: ¬HasFur(Rex)

--- Resolution Trace ---

Step 1: Resolve C6 and C3
  Unification MGU : {?x3 / Rex}
  Derived Clause  : C7: ¬Mammal(Rex) ∨ GivesMilk(Rex)

Step 2: Resolve C7 and C5
  Unification MGU : {}
  Derived Clause  : C8: ¬Mammal(Rex)

Step 3: Resolve C8 and C1
  Unification MGU : {?x1 / Rex}
  Derived Clause  : C9: ¬Dog(Rex)

Step 4: Resolve C9 and C4
  Unification MGU : {}
  Derived Clause  : C10: [] (Empty Clause)
```

5Q)

Domain: University Scholarship & Honors System

1. Knowledge Base (Horn Clauses)

Rules (R_1 to R_5):

- R_1: HighGPA ∧ ResearchPaper → DeanList
- R_2: DeanList ∧ CommunityService → MeritScholarship
- R_3: MeritScholarship ∧ FacultyRecommendation → PresidentialAward
- R_4: AthleticExcellence ∧ PassingGPA → SportsScholarship
- R_5: SportsScholarship ∧ CommunityService → StudentLeadershipGrant

Code:

```
class HornKB:
    def __init__(self):
        self.rules = []  # list of tuples: (list_of_premises, conclusion, rule_name)
        self.facts = set()

    def add_fact(self, fact):
        self.facts.add(fact)

    def add_rule(self, premises, conclusion, name=""):
        self.rules.append((premises, conclusion, name))
def forward_chaining(kb, query):
    print("=" * 65)
    print("          FORWARD CHAINING INFERENCE TRACE            ")
    print("=" * 65)
    count = {name: len(premises) for premises, _, name in kb.rules}
    inferred = {name: False for name in count}
    known_facts = set(kb.facts)
    agenda = list(kb.facts)
    iteration = 1
    print(f"Initial Facts: {sorted(list(known_facts))}\n")
    while agenda:
        p = agenda.pop(0)
        print(f"[Iteration {iteration}] Processing fact: '{p}'")
        iteration += 1
        for premises, conclusion, name in kb.rules:
            if p in premises:
                count[name] -= 1
                if count[name] == 0 and not inferred[name]:
                    inferred[name] = True
                    print(f"  -> Rule {name} FIRED: {' ∧ '.join(premises)} → {conclusion}")
                    if conclusion == query:
                        print(f"  -> QUERY '{query}' REACHED!")
                        return True
                    if conclusion not in known_facts:
                        known_facts.add(conclusion)
                        agenda.append(conclusion)
                        print(f"  -> New Fact Inferred: '{conclusion}'")
        print(f"  Current Known Facts: {sorted(list(known_facts))}\n")

    return query in known_facts
def backward_chaining(kb, goal, depth=0, visited=None):
    if visited is None:
        visited = set()
```

```
    indent = "  " * depth
    print(f"{indent}?- Goal: [{goal}]")
    if goal in kb.facts:
        print(f"{indent}
        return True

    # Cycle check
    if goal in visited:
        print(f"{indent}  :'{goal}'")
        return False
    visited.add(goal)
    candidate_rules = [(prems, concl, name) for prems, concl, name in kb.rules if concl ==
goal]
    if not candidate_rules:
        print(f"{indent}  ✕ No rules conclude '{goal}' and not in facts.")
        return False
    for premises, conclusion, name in candidate_rules:
        print(f"{indent}  -> Trying Rule {name}: {' ∧ '.join(premises)} → {conclusion} (AND
node)")
        all_subgoals_proven = True
        for premise in premises:
            if not backward_chaining(kb, premise, depth + 2, visited.copy()):
                all_subgoals_proven = False
                break
        if all_subgoals_proven:
            print(f"{indent}  ✓ Rule {name} SUCCEEDED. Proved goal: '{goal}'")
            return True

    print(f"{indent}  ✕ Failed to prove goal: '{goal}'")
    return False
kb.add_fact("HighGPA")
kb.add_fact("ResearchPaper")
kb.add_fact("CommunityService")
kb.add_fact("FacultyRecommendation")
kb.add_rule(["HighGPA", "ResearchPaper"], "DeanList", "R1")
kb.add_rule(["DeanList", "CommunityService"], "MeritScholarship", "R2")
kb.add_rule(["MeritScholarship", "FacultyRecommendation"], "PresidentialAward", "R3")
kb.add_rule(["AthleticExcellence", "PassingGPA"], "SportsScholarship", "R4")
kb.add_rule(["SportsScholarship", "CommunityService"], "StudentLeadershipGrant", "R5")

target_query = "PresidentialAward"

# Run Forward Chaining
fc_result = forward_chaining(kb, target_query)

# Run Backward Chaining
print("\n" + "=" * 65)
print("          BACKWARD CHAINING INFERENCE TRACE          ")
print("=" * 65)
bc_result = backward_chaining(kb, target_query)
```

print(f"\nFinal Backward Chaining Result: {bc_result}")

```
                BACKWARD CHAINING INFERENCE TRACE
============================================================
?- Goal: [PresidentialAward]
  -> Trying Rule R3: MeritScholarship ∧ FacultyRecommendation → PresidentialAwa
rd (AND node)
    ?- Goal: [MeritScholarship]
      -> Trying Rule R2: DeanList ∧ CommunityService → MeritScholarship (AND no
de)
        ?- Goal: [DeanList]
          -> Trying Rule R1: HighGPA ∧ ResearchPaper → DeanList (AND node)
            ?- Goal: [HighGPA]
              ✔ Matched Known Ground Fact: 'HighGPA'
            ?- Goal: [ResearchPaper]
              ✔ Matched Known Ground Fact: 'ResearchPaper'
          ✔ Rule R1 SUCCEEDED. Proved goal: 'DeanList'
        ?- Goal: [CommunityService]
          ✔ Matched Known Ground Fact: 'CommunityService'
      ✔ Rule R2 SUCCEEDED. Proved goal: 'MeritScholarship'
    ?- Goal: [FacultyRecommendation]
      ✔ Matched Known Ground Fact: 'FacultyRecommendation'
  ✔ Rule R3 SUCCEEDED. Proved goal: 'PresidentialAward'

Final Backward Chaining Result: True
```

Comparison of Forward and Backward Chaining

Dimension	Forward Chaining (FC)	Backward Chaining (BC)
Reasoning Direction	Data-driven (Bottom-up): Starts with known atomic facts and applies rules forward to generate all inferable conclusions.	Goal-driven (Top-down): Starts with the target query and works backward through sub-goals to ground facts.
Search Space & Efficiency	May infer many irrelevant facts that do not contribute to proving the target query (e.g., if there were many unrelated rules triggered by the initial facts).	Explores only the sub-trees and premises directly relevant to answering the query, avoiding irrelevant inferences.
Time & Space Complexity	Runs in linear time $O(n)$with respect to the size of the Horn KB when using counter-based tracking. Space is bounded by the total number of facts.	Worst-case exponential $O(b^d)$if branching factor b(multiple rules for same goal) and depth $d$are large, but linear space along recursion depth.
Handling Incomplete Data	Requires all initial facts up front to derive new conclusions.	Can prompt for missing facts dynamically only when a sub-goal needs verification.
Suitability for this Problem	Suitable if the system aims to precompute all possible awards/certifications for a student whenever their record updates.	Highly preferred when verifying a single specific query (PresidentialAward), as it skips unrelated rules like $R_4$and R_5completely.